

MQTT-Based Remote HVAC Monitoring System

1. Project Overview

The **MQTT-Based Remote HVAC Monitoring System** is an IoT-enabled system designed to monitor the operating conditions of **Heating, Ventilation, and Air Conditioning (HVAC)** equipment remotely. HVAC systems are widely used in industries, offices, hospitals, laboratories, shopping malls, data centers, and residential buildings.

Continuous monitoring of temperature, humidity, air quality, and equipment status can help maintain comfortable environmental conditions while also identifying abnormal operating conditions at an early stage.

The proposed system uses an **ESP8266 NodeMCU** as the main controller. Sensors measure parameters such as room temperature, humidity, and air quality. The ESP8266 processes the sensor data and publishes it to an **MQTT broker** through Wi-Fi. A remote MQTT client or dashboard subscribes to the relevant topics and displays the HVAC parameters in real time.

2. Problem Statement

HVAC systems must operate within appropriate temperature and humidity conditions to maintain occupant comfort and protect equipment. Conventional HVAC monitoring often depends on local displays or manual inspection.

If temperature, humidity, or air quality becomes abnormal, the problem may not be noticed immediately. In addition, HVAC equipment may continue operating unnecessarily, increasing energy consumption.

Another challenge is that maintenance personnel may not always be physically present near the HVAC system. Therefore, a remote monitoring system is required to provide real-time information from the HVAC environment.

3. Proposed Solution

The proposed system uses an **ESP8266 NodeMCU with MQTT communication** to monitor HVAC environmental conditions remotely.

A DHT11/DHT22 sensor measures temperature and humidity, while an MQ-135 sensor can be used as a low-cost prototype indicator for changes in air quality. A relay or status input can represent HVAC ON/OFF operation.

The ESP8266 collects the sensor data, processes it, and publishes the information to an MQTT broker over Wi-Fi.

4. System Architecture

The system is divided into five major layers.

4.1 Sensing Layer

The sensing layer collects environmental information.

Typical sensors:

- DHT11/DHT22 – temperature and humidity
- MQ-135 – prototype air-quality indication
- Current/status sensor – HVAC operating status

4.2 Processing Layer

The ESP8266 NodeMCU acts as the main processing unit.

Its functions include:

- Sensor data acquisition
- Data processing
- Threshold comparison
- HVAC status monitoring
- Alert generation
- MQTT communication

4.3 Communication Layer

The ESP8266 connects to the local Wi-Fi network and communicates with the MQTT broker.

MQTT is lightweight and suitable for IoT applications because it uses a publish/subscribe communication model.

4.4 MQTT Broker Layer

The MQTT broker receives messages published by the ESP8266 and forwards them to subscribed clients.

Examples include:

- Mosquitto
- HiveMQ
- EMQX

- Other MQTT-compatible brokers

4.5 Monitoring Layer

A computer, mobile application, or web dashboard subscribes to MQTT topics and displays the HVAC information.

5. Circuit Table

The following connection table is suitable for an **ESP8266 NodeMCU prototype**.

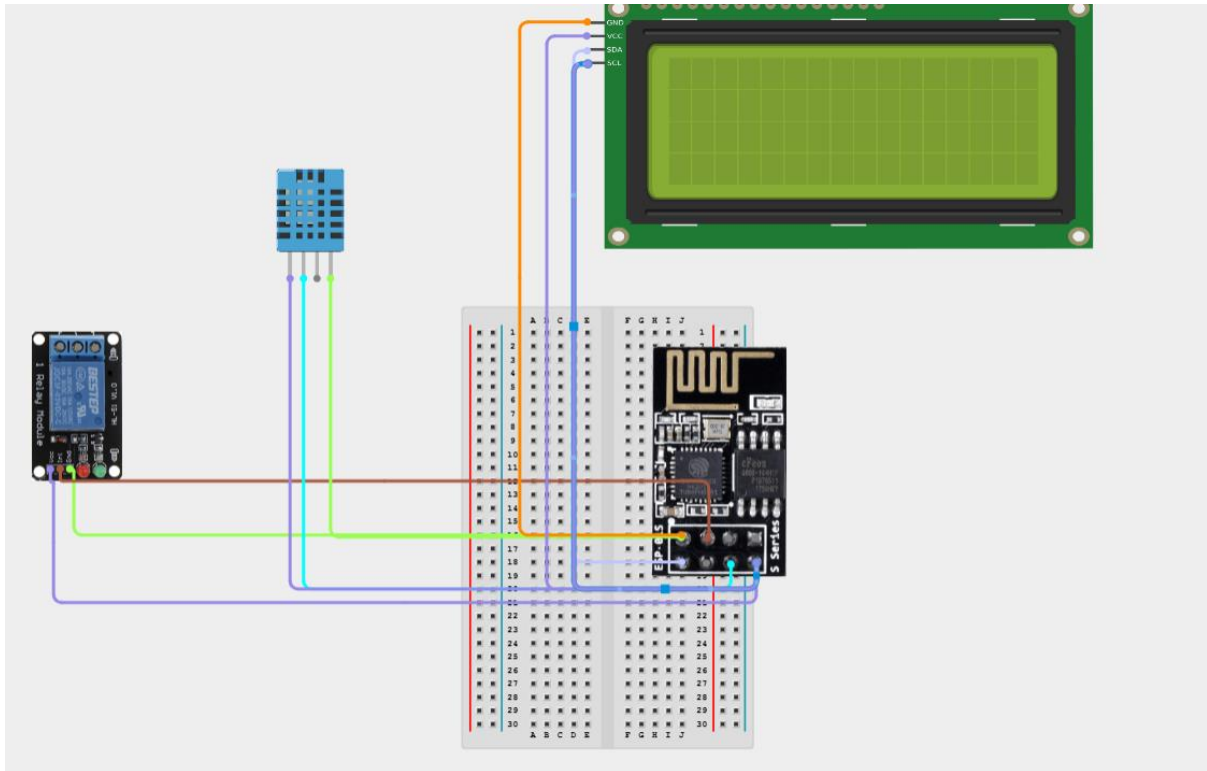
Component	Pin	ESP8266 Pin	Function
DHT11/DHT22	DATA	D3 / GPIO0	Temperature & humidity
	VCC	3.3V	Power
	GND	GND	Ground
MQ-135	AO	A0*	Air-quality indication
	VCC	5V/VU*	Power
	GND	GND	Ground
HVAC Status Input	Signal	D5 / GPIO14	HVAC ON/OFF
Buzzer	+	D7 / GPIO13	Warning alarm
	–	GND	Ground
OLED I ² C	SDA	D2 / GPIO4	Display data
	SCL	D1 / GPIO5	Display clock
	VCC	3.3V	Power
	GND	GND	Ground

Important

The ESP8266 has only **one analog input A0**. Therefore, if multiple analog sensors are required, an external ADC such as an **ADS1115** should be used.

Also, the MQ-135 output must be appropriately scaled/conditioned so that the voltage applied to the ESP8266 A0 input remains within its safe input range.

6. Circuit Design



Circuit Design:

The proposed circuit is designed around an ESP32 microcontroller, which acts as the central processing unit. The pulse sensor provides an analog signal corresponding to the user's pulse rate. The gas sensor detects the presence or variation of gases and provides an analog output to the controller. The DHT11 sensor measures environmental temperature and humidity and sends digital data to the ESP32. A buzzer is connected to a digital output and is activated whenever a predefined abnormal condition is detected.

All sensors share a common ground with the ESP32. The ESP32 continuously reads the sensor values, processes the measurements, compares them with predefined threshold values, and generates an alarm through the buzzer when necessary.

7. Algorithm

- **Step-by-step algorithm**
- Start the ESP8266.
- Initialize the DHT sensor.
- Initialize the MQ-135 input.
- Initialize OLED and buzzer.
- Connect to Wi-Fi.
- Connect to the MQTT broker.
- Read temperature.
- Read humidity.
- Read air-quality sensor value.
- Read HVAC operating status.

- Compare readings with predefined limits.
- Determine HVAC condition.
- Display values on OLED.
- Activate buzzer if necessary.
- Publish data to MQTT topics.
- Repeat continuously.

8. Coding

The following is a complete **ESP8266 MQTT prototype program**.

```
#include <ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
#include <DHT.h>
```

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
// =====
```

```
// WIFI SETTINGS
```

```
// =====
```

```
const char* WIFI_SSID = "YOUR_WIFI_NAME";
```

```
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
```

```
// =====
```

```
// MQTT SETTINGS
```

```
// =====
```

```
// Replace with your MQTT broker address
const char* MQTT_SERVER = "YOUR_MQTT_BROKER";

const int MQTT_PORT = 1883;

// If your broker requires login
const char* MQTT_USER = "";
const char* MQTT_PASSWORD = "";

// =====
// MQTT TOPICS
// =====

const char* TOPIC_TEMPERATURE =
    "hvac/temperature";

const char* TOPIC_HUMIDITY =
    "hvac/humidity";

const char* TOPIC_AIR =
    "hvac/airquality";

const char* TOPIC_STATUS =
    "hvac/status";

const char* TOPIC_ALERT =
    "hvac/alert";
```

```
// =====  
// PIN DEFINITIONS  
// =====
```

```
#define DHT_PIN D3  
#define DHT_TYPE DHT11
```

```
#define MQ135_PIN A0
```

```
#define HVAC_STATUS_PIN D5
```

```
#define BUZZER_PIN D7
```

```
#define OLED_SDA D2  
#define OLED_SCL D1
```

```
// =====  
// OBJECTS  
// =====
```

```
DHT dht(  
  DHT_PIN,  
  DHT_TYPE  
);
```

```
WiFiClient espClient;
```

```
PubSubClient mqttClient(  
    espClient  
);
```

```
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64
```

```
Adafruit_SSD1306 display(  
    SCREEN_WIDTH,  
    SCREEN_HEIGHT,  
    &Wire,  
    -1  
);
```

```
// =====  
// VARIABLES  
// =====
```

```
float temperature = 0.0;
```

```
float humidity = 0.0;
```

```
int airQuality = 0;
```

```
int hvacStatus = 0;
```

```
String systemStatus = "NORMAL";
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(115200);
```

```
  Serial.println();
```

```
  Serial.println(
```

```
    "MQTT HVAC MONITORING SYSTEM"
```

```
  );
```

```
// -----
```

```
// PIN CONFIGURATION
```

```
// -----
```

```
pinMode(
```

```
  HVAC_STATUS_PIN,
```

```
  INPUT_PULLUP
```

```
);
```

```
pinMode(
```

```
BUZZER_PIN,  
OUTPUT  
);
```

```
digitalWrite(  
    BUZZER_PIN,  
    LOW  
);
```

```
// -----  
// DHT  
// -----
```

```
dht.begin();
```

```
// -----  
// OLED  
// -----
```

```
Wire.begin(  
    OLED_SDA,  
    OLED_SCL  
);
```

```
if (  
    !display.begin(  
        128, 64,  
        0x3C, 0x3D, 0x3E, 0x3F,  
        0x2D, 0x2E, 0x2F, 0x28, 0x29, 0x2A, 0x2B, 0x2C,  
        0x25, 0x26, 0x27, 0x22, 0x23, 0x24, 0x21, 0x02, 0x03, 0x04, 0x05,  
        0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F
```

```
    SSD1306_SWITCHCAPVCC,  
    0x3C  
  )  
)  
{  
  Serial.println(  
    "OLED initialization failed"  
  );  
}  
else  
{  
  display.clearDisplay();  
  
  display.setTextSize(1);  
  
  display.setTextColor(  
    SSD1306_WHITE  
  );  
  
  display.setCursor(0, 0);  
  
  display.println(  
    "MQTT HVAC MONITOR"  
  );  
  
  display.setCursor(0, 15);  
  
  display.println(  
    "Starting..."
```

```
);

display.display();

delay(2000);
}

// -----
// WIFI
// -----

connectWiFi();

// -----
// MQTT
// -----

mqttClient.setServer(
  MQTT_SERVER,
  MQTT_PORT
);
}

// =====
// MAIN LOOP
// =====
```

```
void loop()
{

    // Reconnect Wi-Fi if required

    if (
        WiFi.status() != WL_CONNECTED
    )
    {
        connectWiFi();
    }

    // Reconnect MQTT if required

    if (
        !mqttClient.connected()
    )
    {
        connectMQTT();
    }

    mqttClient.loop();

    // -----
    // READ SENSORS
```

```
// -----
```

```
readTemperatureHumidity();
```

```
readAirQuality();
```

```
readHVACStatus();
```

```
// -----
```

```
// ANALYZE
```

```
// -----
```

```
checkHVACCondition();
```

```
// -----
```

```
// LOCAL DISPLAY
```

```
// -----
```

```
displayData();
```

```
// -----
```

```
// SERIAL MONITOR
```

```
// -----
```

```
printData();
```

```
// -----  
  
// MQTT PUBLISH  
  
// -----  
  
publishData();  
  
  
// -----  
  
// WAIT  
  
// -----  
  
delay(10000);  
}  
  
  
// =====  
// WIFI CONNECTION  
// =====  
  
void connectWiFi()  
{  
  
    if (  
        WiFi.status() ==  
        WL_CONNECTED  
    )  
    {  
        return;  
    }  
}
```

```
}
```

```
Serial.print(  
  "Connecting to Wi-Fi"  
);
```

```
WiFi.mode(  
  WIFI_STA  
);
```

```
WiFi.begin(  
  WIFI_SSID,  
  WIFI_PASSWORD  
);
```

```
int count = 0;
```

```
while (  
  WiFi.status() != WL_CONNECTED &&  
  count < 30  
)  
{  
  
  delay(500);
```

```
Serial.print(".");
```

```
count++;
```

```
}
```

```
Serial.println();
```

```
if (
```

```
    WiFi.status() ==
```

```
    WL_CONNECTED
```

```
)
```

```
{
```

```
    Serial.println(
```

```
        "Wi-Fi connected"
```

```
    );
```

```
    Serial.print(
```

```
        "IP Address: "
```

```
    );
```

```
    Serial.println(
```

```
        WiFi.localIP()
```

```
    );
```

```
}
```

```
else
{

    Serial.println(
        "Wi-Fi connection failed"
    );

}
}

// =====
// MQTT CONNECTION
// =====

void connectMQTT()
{

    while (
        !mqttClient.connected()
    )
    {

        Serial.print(
            "Connecting to MQTT..."
        );

        String clientID =
```

```
"ESP8266-HVAC-" +
```

```
String(
```

```
    ESP.getChipId()
```

```
);
```

```
bool connected;
```

```
if (
```

```
    strlen(MQTT_USER) == 0
```

```
)
```

```
{
```

```
    connected =
```

```
        mqttClient.connect(
```

```
            clientID.c_str()
```

```
        );
```

```
}
```

```
else
```

```
{
```

```
    connected =
```

```
        mqttClient.connect(
```

```
            clientID.c_str(),
```

```
            MQTT_USER,
```

```
            MQTT_PASSWORD
```

```
        );
```

```
}
```

```
if (connected)
```

```
{
```

```
    Serial.println(
```

```
        "connected"
```

```
    );
```

```
    // Publish online status
```

```
    mqttClient.publish(
```

```
        TOPIC_STATUS,
```

```
        "ONLINE"
```

```
    );
```

```
}
```

```
else
```

```
{
```

```
    Serial.print(
```

```
        "failed, state="
```

```
    );
```

```
    Serial.println(
```

```
        mqttClient.state()
```

```
    );
```

```
        delay(5000);
    }
}
}

// =====
// READ TEMPERATURE & HUMIDITY
// =====

void readTemperatureHumidity()
{

    temperature =
        dht.readTemperature();

    humidity =
        dht.readHumidity();

    if (
        isnan(temperature) ||
        isnan(humidity)
    )
    {

        Serial.println(
            "DHT sensor reading failed"
```

```
);

temperature = 0;

humidity = 0;
}
}

// =====
// READ AIR QUALITY
// =====

void readAirQuality()
{

    airQuality =
        analogRead(
            MQ135_PIN
        );

}

// =====
// READ HVAC STATUS
// =====

void readHVACStatus()
```

```

{

// INPUT_PULLUP:
// LOW  = HVAC ON
// HIGH = HVAC OFF

if (
    digitalRead(
        HVAC_STATUS_PIN
    ) == LOW
)
{
    hvacStatus = 1;
}
else
{
    hvacStatus = 0;
}
}

// =====
// HVAC CONDITION ANALYSIS
// =====

void checkHVACCondition()
{

    bool warning = false;

```

```
bool critical = false;
```

```
// -----
```

```
// HIGH TEMPERATURE
```

```
// -----
```

```
if (
```

```
    temperature > 35
```

```
)
```

```
{
```

```
    critical = true;
```

```
}
```

```
else if (
```

```
    temperature > 30
```

```
)
```

```
{
```

```
    warning = true;
```

```
}
```

```
// -----
```

```
// HIGH HUMIDITY
```

```
// -----
```

```
if (  
    humidity > 85  
)  
{  
  
    warning = true;  
  
}
```

```
// -----  
// AIR QUALITY  
// -----
```

```
// Prototype threshold.  
// Calibrate according to the actual sensor.
```

```
if (  
    airQuality > 800  
)  
{
```

```
    critical = true;
```

```
}
```

```
else if (  
    airQuality > 500  
)
```

```
{

    warning = true;

}

// -----
// FINAL STATUS
// -----

if (critical)
{

    systemStatus =
        "CRITICAL";

    digitalWrite(
        BUZZER_PIN,
        HIGH
    );

}
else if (warning)
{

    systemStatus =
        "WARNING";
```

```
digitalWrite(
    BUZZER_PIN,
    HIGH
);

}
else
{

    systemStatus =
        "NORMAL";

    digitalWrite(
        BUZZER_PIN,
        LOW
    );

}
}

// =====
// OLED DISPLAY
// =====

void displayData()
```

```
{  
  
    display.clearDisplay();  
  
    display.setTextSize(1);  
  
    display.setTextColor(  
        SSD1306_WHITE  
    );  
  
    display.setCursor(0, 0);  
  
    display.println(  
        "HVAC MONITOR"  
    );  
  
    display.setCursor(0, 12);  
  
    display.print(  
        "Temp: "  
    );  
  
    display.print(  
        temperature,  
        1  
    );  
}
```

```
display.println(  
  " C"  
);
```

```
display.setCursor(0, 23);
```

```
display.print(  
  "Humidity: "  
);
```

```
display.print(  
  humidity,  
  1  
);
```

```
display.println(  
  "%"  
);
```

```
display.setCursor(0, 34);
```

```
display.print(  
  "Air: "  
);
```

```
display.println(  
  "
```

```
    airQuality  
);
```

```
display.setCursor(0, 45);
```

```
display.print(  
    "HVAC: "  
);
```

```
if (hvacStatus)  
{  
    display.println(  
        "ON"  
    );  
}  
else  
{  
    display.println(  
        "OFF"  
    );  
}
```

```
display.setCursor(0, 56);
```

```
display.print(  
    "Status: "  
);
```

```
display.println(  
    systemStatus  
);
```

```
display.display();  
}
```

```
// =====  
// SERIAL MONITOR  
// =====
```

```
void printData()  
{
```

```
    Serial.println();
```

```
    Serial.println(  
        "===== HVAC DATA ====="  
    );
```

```
    Serial.print(  
        "Temperature: "  
    );
```

```
    Serial.print(
```

```
    temperature  
);
```

```
Serial.println(  
    " C"  
);
```

```
Serial.print(  
    "Humidity: "  
);
```

```
Serial.print(  
    humidity  
);
```

```
Serial.println(  
    " %"  
);
```

```
Serial.print(  
    "Air Quality ADC: "  
);
```

```
Serial.println(  
    airQuality  
);
```

```
Serial.print(  
  "HVAC Status: "  
);
```

```
if (hvacStatus)  
{  
  Serial.println(  
    "ON"  
  );  
}  
else  
{  
  Serial.println(  
    "OFF"  
  );  
}
```

```
Serial.print(  
  "System Status: "  
);
```

```
Serial.println(  
  systemStatus  
);
```

```
Serial.println(  
  "
```

```

    "=====
);
}

// =====
// MQTT PUBLISH
// =====

void publishData()
{

    if (
        !mqttClient.connected()
    )
    {
        return;
    }

    char temperatureString[10];

    char humidityString[10];

    char airString[10];

    dtostrf(
        temperature,

```

```
1,  
2,  
    temperatureString  
);
```

```
dtostrf(  
    humidity,  
    1,  
    2,  
    humidityString  
);
```

```
sprintf(  
    airString,  
    "%d",  
    airQuality  
);
```

```
// -----  
// TEMPERATURE  
// -----
```

```
mqttClient.publish(  
    TOPIC_TEMPERATURE,  
    temperatureString  
);
```

```
// -----  
// HUMIDITY  
// -----
```

```
mqttClient.publish(  
    TOPIC_HUMIDITY,  
    humidityString  
);
```

```
// -----  
// AIR QUALITY  
// -----
```

```
mqttClient.publish(  
    TOPIC_AIR,  
    airString  
);
```

```
// -----  
// HVAC STATUS  
// -----
```

```
if (hvacStatus)  
{
```

```
mqttClient.publish(
  TOPIC_STATUS,
  "ON"
);

}
else
{

  mqttClient.publish(
    TOPIC_STATUS,
    "OFF"
  );

}


// -----
// ALERT
// -----

mqttClient.publish(
  TOPIC_ALERT,
  systemStatus.c_str()
);

Serial.println(
  "MQTT data published"
```

```
);  
}
```

9. System Working

When the system is powered on, the ESP8266 initializes all sensors and the OLED display. It then connects to the configured Wi-Fi network.

After obtaining a network connection, the ESP8266 establishes a connection with the MQTT broker.

The DHT11/DHT22 continuously measures temperature and humidity. The MQ-135 provides an analog reading that represents changes in the air-quality environment. The HVAC status input indicates whether the HVAC unit is operating.

The ESP8266 processes these measurements and compares them against configured thresholds.

The system then publishes the data to the MQTT broker.

A remote monitoring application subscribed to the MQTT topics receives the information immediately.

MQTT communication

This allows maintenance personnel to monitor HVAC conditions without being physically present near the equipment.

10. Bill of Materials

S.No.	Component	Quantity	Approx. Cost
1	ESP8266 NodeMCU	1	₹300–₹500
2	DHT11/DHT22	1	₹80–₹250
3	MQ-135 Sensor	1	₹100–₹200
4	0.96" OLED I ² C	1	₹150–₹300
5	Buzzer	1	₹10–₹30
6	Resistors	Set	₹20–₹50

S.No.	Component	Quantity	Approx. Cost
7	Breadboard	1	₹80–₹150
8	Jumper Wires	1 set	₹50–₹150
9	USB Cable	1	₹50–₹150
10	5V/USB Power Supply	1	₹100–₹250

Estimated prototype cost

Approximately ₹940–₹2,030, depending on component quality and supplier.

The MQTT broker can be hosted locally using software such as Mosquitto or through a suitable cloud MQTT service.

11. Prototype Implementation

The prototype can be implemented on a breadboard using the ESP8266 NodeMCU.

Step 1 – ESP8266 setup

Mount the NodeMCU on the breadboard and provide regulated USB power.

Step 2 – Temperature and humidity sensor

Connect the DHT11/DHT22 to D3.

Step 3 – Air-quality sensor

Connect the MQ-135 through appropriate voltage scaling to A0.

Step 4 – HVAC status

Use a switch in the prototype to simulate HVAC ON/OFF status.

Step 5 – OLED

Connect the OLED through the I²C interface:

Step 6 – Buzzer

Connect the buzzer to D7 through an appropriate driver if necessary.

Step 7 – MQTT broker

Configure an MQTT broker and create the required topics.

Step 8 – Program ESP8266

Upload the Arduino program after entering:

Wi-Fi SSID

Wi-Fi Password

MQTT Broker Address

MQTT Port

MQTT Username

MQTT Password

Step 9 – Test communication

Use an MQTT client to subscribe to:

The client should receive all HVAC messages.

Step 10 – Test abnormal conditions

Test:

- Increasing temperature
- Increasing humidity
- Changing air-quality sensor reading
- Turning HVAC status ON/OFF

The system should update the OLED, buzzer, and MQTT dashboard accordingly.

12. Results & Performance Analysis

The proposed system can continuously monitor HVAC environmental parameters and communicate the measurements to a remote MQTT client.

Expected results

Parameter	Normal Condition	Abnormal Condition	System Response
Temperature	Comfortable range	Excessive temperature	Warning/Alarm
Humidity	Normal range	High humidity	Warning
Air Quality	Low sensor reading	High sensor reading	Warning/Critical
HVAC Status	ON/OFF	Unexpected condition	MQTT update

Parameter	Normal Condition	Abnormal Condition	System Response
Wi-Fi	Connected	Disconnected	Reconnection attempt
MQTT	Connected	Connection lost	Automatic reconnect
OLED	Displays readings	Displays warning	Local indication
Buzzer	OFF	Abnormal condition	ON

12.1 Temperature Monitoring Performance

The DHT11/DHT22 provides continuous temperature information. When the temperature exceeds the configured threshold, the controller changes the system status and publishes the warning through MQTT.

12.2 Humidity Monitoring Performance

Humidity is monitored continuously. Excessive humidity can indicate inadequate ventilation, cooling problems, or environmental conditions that may affect equipment and occupant comfort.

12.3 Air Quality Monitoring Performance

The MQ-135 provides an analog indication of changes in air quality. A higher sensor reading can be used to trigger a warning in the prototype.

For an actual HVAC installation, a calibrated air-quality sensor should be used if quantitative air-quality measurements are required.

12.4 MQTT Communication Performance

MQTT provides lightweight communication between the ESP8266 and remote clients. Because the system uses topics, multiple monitoring applications can subscribe to the same HVAC data without requiring the ESP8266 to communicate separately with every client.

12.5 Remote Monitoring

The system allows maintenance personnel to monitor HVAC conditions remotely. This reduces dependence on physical inspection and can improve response time when abnormal conditions occur.

Conclusion

The **MQTT-Based Remote HVAC Monitoring System** provides an efficient IoT solution for monitoring HVAC environmental conditions remotely. The ESP8266 acts as the central controller and collects temperature, humidity, air-quality, and HVAC status information.

The collected information is displayed locally through an OLED and transmitted using the **MQTT publish/subscribe protocol**. A remote MQTT client can subscribe to the relevant topics and monitor the HVAC system from another location.

The system can help identify abnormal temperature, excessive humidity, and poor air-quality conditions at an early stage. It also provides a foundation for **remote maintenance, energy monitoring, fault detection, and smart-building automation**.

Future enhancements

The project can be further improved by adding:

- CO₂ sensor
- PM2.5 sensor
- Current/power sensor
- Compressor monitoring
- Fan-speed monitoring
- Refrigerant-pressure monitoring
- Energy-consumption measurement
- Mobile notifications
- Web dashboard
- Automatic HVAC control
- Data logging

OUTPUT:

